Department of Information Technology
V R Siddhartha Engineering College

SaraswathAI : An **Agentic** Blockchain-Integrated Domain-Specific LLM Platform for Optimized Educational Content Retrieval, Personalized Learning, and **Intelligent Assessment Education**

B.Tech in Information Technology
Major Project Review Presentation

Presented by

Ms. Neeraja Gurram (228W1A12F5)

Mr. Lokesh Narne (228W1A12H4)

Under the guidance of

Dr. Ramesh Mande , Senior Asst.Prof.

Agenda

- Introduction & Problem Statement
- Objectives and Outcomes of the Project
- Literature Review and Summary
- Datasets and Requirement Specifications
- Algorithms used in the Project
- Architecture Diagram
- Implementation steps & Execution
- Results obtained
- Result Analysis
- Conclusion & Future Work
- References

Introduction

- Educational ecosystems are rapidly evolving through digital platforms that enable flexible and personalized learning experiences. Intelligent learning systems increasingly assist students in accessing academic resources beyond traditional classroom environments.
- As reliance on automated learning grows, education is shifting toward continuous, self-directed, and outcome-oriented learning models requiring sustained learner engagement.

Problem Statement

- Despite improved access to educational content, existing learning platforms largely operate as passive information retrieval systems without monitoring student understanding or progress.
- The lack of **structured practice, evaluation**, and feedback limits effective **knowledge retention** and personalized improvement. This creates a need for systems that support continuous learning assessment and adaptive academic guidance.

Objectives

- To extend the SaraswathAI platform with an agentic framework capable of autonomously managing learning workflows beyond query-based interaction.
- To design mechanisms for automatic quiz generation based on curriculum-specific retrieved educational content.
- To implement intelligent evaluation modules that analyze student responses and provide adaptive academic feedback.
- To integrate assessment-driven learning with the existing secure and personalized educational content retrieval system.

Outcomes

- Students will receive continuous learning support through automated practice and performance evaluation after content interaction.
- The system will enable adaptive learning experiences by identifying knowledge gaps and guiding personalized improvement.
- Learning effectiveness and knowledge retention will improve through structured assessment and feedback mechanisms.
- The platform will evolve from a passive information provider into an intelligent and self-guided learning environment.

Literature Survey

Paper Title	Year	Dataset	Review
A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems	2025	Survey (no specific dataset; covers diverse benchmarks across biomedicine, programming, finance domains)	This survey reviews self-evolving AI agent techniques where LLM-based agents autonomously improve through interaction data and environmental feedback. A unified framework with four components — System Inputs, Agent System, Environment, and Optimisers — is proposed. Domain-specific evolution strategies and evaluation methods are systematically examined. Safety and ethical considerations for lifelong agentic systems are also addressed.
LLM-Based Educational Content Retrieval and Intelligent Query Processing (Information Processing & Management, 2026)	2026	Curriculum-specific educational document corpora; benchmark IR datasets	This article addresses LLM-driven information retrieval with domain-specific optimizations for educational content. RAG pipelines and neural indexing strategies for academic resources are examined. Query understanding and response quality in knowledge-intensive scenarios are evaluated, contributing foundational techniques for building intelligent, domain-aware retrieval systems.
AI-Driven Learning Interaction and Engagement Framework for Digital Education (Open Access Ed-Tech Journal, 2026)	2026	Student interaction logs; digital learning platform usage data	This paper proposes an AI framework to analyze and optimize learner engagement in digital educational platforms. Automated systems track student interactions, identify disengagement patterns, and trigger adaptive interventions. Integration of AI agents with learning management systems for continuous performance monitoring is explored, with results showing improved learner retention through intelligent feedback loops.

Literature Survey

Paper Title	Year	Dataset	Review
Personalized Learning Pathways and AI-Guided Academic Improvement in Higher Education	2026	Higher education student performance records; learning analytics datasets	This study explores AI-driven personalized learning pathways that adapt content delivery and practice exercises based on individual student performance. Intelligent tutoring systems dynamically adjust difficulty and provide targeted recommendations to address knowledge gaps. Longitudinal analysis confirms improved academic outcomes when adaptive learning is integrated with structured assessments.
Agent4EDU: Advancing AI for Education with Agentic Workflows	2025	Multi-domain educational task benchmarks; student interaction simulation environments	This paper introduces the Agent4EDU framework with four AI agent application models in education: human-AI collaboration, AI assistant, instruction execution, and general type. Built on LLMs and agentic workflows, it enables complex educational tasks including content curation, tutoring, and assessment. Swarm intelligence through multi-agent collaboration is explored as an emergent capability for adaptive education.

Models Used:

•**SOTA Model**: llama3.2,llama 3, qwenq-2.5,phi-4,dolphin-mistral,phi-3.5.

•**Embedding models** : sentence-transformers/all-mpnet-base-v2, sentence-transformers/all-MiniLM-L6-v2.

Requirements

- **Programming Language**: Python
- **Libraries**:

  LangChain: Builds LLM-powered chains for topic extraction, quiz generation, and feedback.

  PyPDF2: Extracts text from uploaded PDF documents page by page.

  NetworkX: Builds and analyzes the knowledge graph structure and learning paths.

  Plotly: Renders interactive visual representations of the knowledge graph.

  FAISS / ChromaDB: Stores and retrieves document embeddings for similarity-based search.

- **Backend**:

  Ollama: Locally serves and manages SOTA models like Llama, Phi, and Mistral.

  Streamlit: Builds the interactive web interface for the entire application.

Algorithms used in the Project

- **Betweenness Centrality Algorithm:** A network analysis algorithm that measures how often a node appears on the shortest path between other nodes. It is used in the Knowledge Graph to identify the most important and connecting topics a student should focus on.
- **Topological Sort (NetworkX):** A graph algorithm that linearly orders nodes based on their dependencies. It is used to determine the optimal sequence of topics in the learning path, ensuring prerequisites are studied before advanced concepts.
- **Spring Layout Algorithm (Force-Directed):** A graph visualization algorithm that positions nodes by simulating attraction and repulsion forces between them. It is used in the Knowledge Graph to automatically arrange topic nodes into a clear, readable, and well-spaced visual layout.
- **Mastery-Based Progression Algorithm:** An algorithm that tracks and updates a learner's knowledge level using a diminishing returns formula. It is used in the Student Model to categorize topics as mastered, in-progress, or to-learn based on study time and quiz performance.
- **Adaptive Difficulty Algorithm:** An algorithm that dynamically adjusts challenge levels based on a user's current performance. It is used in the Quiz Generator to set question difficulty to easy, medium, or hard depending on the student's average mastery score.

Algorithm

```
SARASWATHAI_LEARNING_SYSTEM():

INPUT:
    Knowledge Graph G(V, E)
    Learner L
    Content Database D

OUTPUT:
    Personalized Learning Path + Adaptive Learning Experience

1. // STEP 1: Knowledge Graph Processing
   centrality_scores = betweenness_centrality(G)
   important_topics = SELECT_TOP_NODES(centrality_scores)

2. // STEP 2: Learning Path Generation
   learning_path = TOPOLOGICAL_SORT(G)

3. // STEP 3: Graph Visualization
   positions = FORCE_DIRECTED_LAYOUT(G)
   DISPLAY_GRAPH(G, positions)

4. // STEP 4: Learning + Assessment Loop

   FOR each topic t in learning_path:

       difficulty = INITIAL_DIFFICULTY

       MASTERY_BASED_PROGRESSION(L, [t], THRESHOLD)

       WHILE mastery not achieved for topic t:
           // Content Retrieval
           content = RETRIEVE_CONTENT(D, t)
           PRESENT(content)

           // Quiz Generation
           quiz = GENERATE_QUIZ(content, difficulty)

           // Student Evaluation
           performance, response_time = EVALUATE(L, quiz)

           // STEP 5: Adaptive Difficulty
           difficulty = ADAPTIVE_DIFFICULTY(
                        difficulty,
                        performance,
                        response_time
                      )

           // Feedback
           PROVIDE_FEEDBACK(L, performance)

       END WHILE

       MARK_TOPIC_COMPLETE(t)

   END FOR

5. RETURN "Personalized Learning Completed"
```

1. Betweenness Centrality Algorithm

The betweenness centrality of a node v is given by the expression:

$$g(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from node s to node t and $\sigma_{st}(v)$ is the number of those paths that pass through v (not where v is an end point).[2]

```
def betweenness_centrality(G, k=None, normalized=True, weight=None,
endpoints=False, seed=None):

    Parameters
    ----------
    G : graph A NetworkX graph.
    k : int, optional (default=None)
    If k is not None use k node samples to estimate betweenness.
    The value of k <= n where n is the number of nodes in the graph.
    Higher values give better approximation.
    normalized : bool, optional
    If True the betweenness values are normalized by `2/((n-1)(n-2))`
    for graphs, and `1/((n-1)(n-2))` for directed graphs where `n`
    is the number of nodes in G.
    weight : None or string, optional (default=None)
    If None, all edge weights are considered equal.
    Otherwise holds the name of the edge attribute used as weight.
    endpoints : bool, optional
    If True include the endpoints in the shortest path counts.

    Returns
    -------
    nodes : dictionary
    Dictionary of nodes with betweenness centrality as the value.
```

```
 betweenness = dict.fromkeys(G, 0.0) # b[v]=0 for v in G
   if k is None:
      nodes = G
   else:
      random.seed(seed)
      nodes = random.sample(G.nodes(), k)
   for s in nodes:

      # single source shortest paths
      if weight is None: # use BFS
         S, P, sigma = _single_source_shortest_path_basic(G, s)
      else: # use Dijkstra's algorithm
         S, P, sigma = _single_source_dijkstra_path_basic(G, s,
weight)

      # accumulation
      if endpoints:
         betweenness = _accumulate_endpoints(betweenness, S, P,
sigma, s)
      else:
         betweenness = _accumulate_basic(betweenness, S, P,
sigma, s)

   # rescaling
   betweenness = _rescale(betweenness, len(G),
normalized=normalized,
                  directed=G.is_directed(), k=k)
   return betweenness
```

2. Topological Sort (NetworkX)

```
TOPOLOGICAL_SORT(G):
  INPUT: Directed graph G = (V, E)
  OUTPUT: List of nodes in topological order

  1. Create a dictionary IN_DEGREE for all vertices
    For each vertex v in V:
      IN_DEGREE[v] = 0

  2. Compute in-degrees
    For each edge (u → v) in E:
      IN_DEGREE[v] += 1

  3. Initialize a queue Q
    For each vertex v in V:
      If IN_DEGREE[v] == 0:
        Enqueue v into Q

  4. Initialize empty list TOPO_ORDER

  5. While Q is not empty:
      u = Dequeue from Q
      Add u to TOPO_ORDER

      For each neighbor v of u:
        IN_DEGREE[v] -= 1
        If IN_DEGREE[v] == 0:
          Enqueue v into Q

  6. If size of TOPO_ORDER ≠ number of vertices:
      Report error (Graph has a cycle)
  7. Return TOPO_ORDER
```

3. Spring Layout Algorithm (Force-Directed)

```
FORCE_DIRECTED_LAYOUT(G):

    INPUT: Graph G = (V, E)
    OUTPUT: Positions of nodes in 2D space

    1. Initialize positions:
       For each node v in V:
           position[v] = random position

    2. Define constants:
       k = optimal distance between nodes
       temperature = initial movement limit

    3. REPEAT until convergence or max iterations:

       // Reset forces
       For each node v in V:
           disp[v] = (0, 0)

       // Repulsive forces (between all node pairs)
       For each pair of nodes (u, v), u ≠ v:
           delta = position[v] - position[u]
           distance = ||delta||
           force = (k^2) / distance
           disp[v] += (delta / distance) * force
           disp[u] -= (delta / distance) * force

       // Attractive forces (along edges)
       For each edge (u, v) in E:
           delta = position[v] - position[u]
           distance = ||delta||
           force = (distance^2) / k
           disp[v] -= (delta / distance) * force
           disp[u] += (delta / distance) * force

       // Update positions
       For each node v in V:
           move = min(||disp[v]||, temperature)
           position[v] += (disp[v] / ||disp[v]||) * move

       // Cool down (reduce movement over time)
       temperature = COOL(temperature)

    4. RETURN position
```

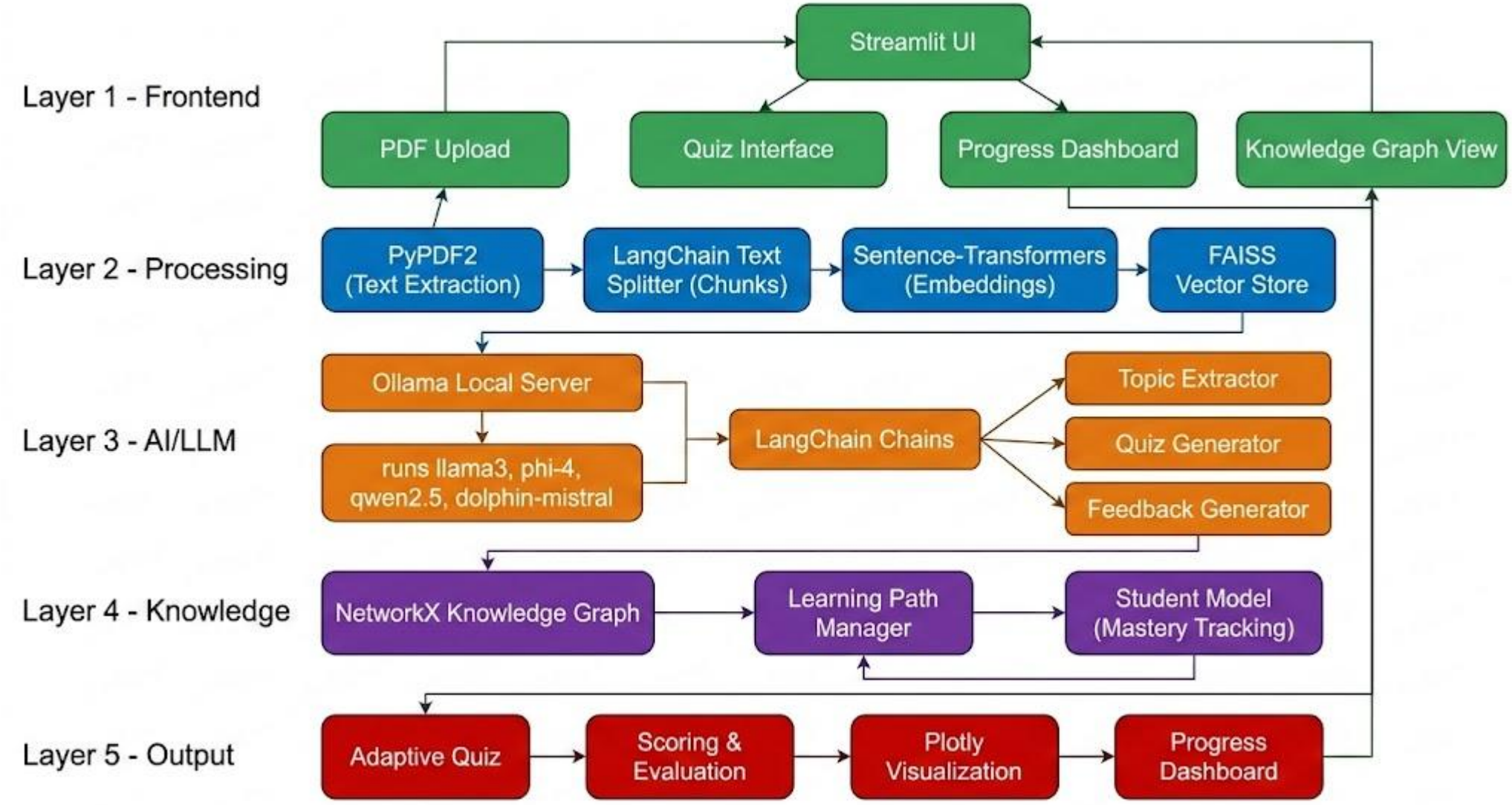
4. Mastery-Based Progression Algorithm

```
MASTERY_BASED_PROGRESSION():
  INPUT:
    Learner L
    Ordered list of levels/skills S = [s1, s2, ..., sn]
    Mastery threshold T (e.g., 80%)

  FOR each skill s in S:

    mastery_score = 0

    WHILE mastery_score < T:

      1. Present learning material or task for skill s
      2. Assess performance:
         result = EVALUATE(L, s)
      3. Update mastery score:
         mastery_score = UPDATE_SCORE(mastery_score, result)
      4. Provide feedback:
         - hints
         - corrections
         - additional practice

    END WHILE

    // Mastery achieved → move forward
    Unlock next skill

  END FOR

  RETURN "All skills mastered"
```

5. Adaptive Difficulty Algorithm

```
ADAPTIVE_DIFFICULTY():

    INPUT:
        current_difficulty  // e.g., 1 (easy) to 10 (hard)
        user_performance    // recent performance score (0 to 1)
        response_time       // time taken by user
        target_performance  // desired accuracy (e.g., 0.75)

    CONSTANTS:
        INCREASE_STEP
        DECREASE_STEP
        MIN_DIFFICULTY
        MAX_DIFFICULTY
        FAST_THRESHOLD
        SLOW_THRESHOLD

1. // Adjust based on accuracy
    IF user_performance > target_performance:
        difficulty = current_difficulty + INCREASE_STEP

    ELSE IF user_performance < target_performance:
        difficulty = current_difficulty - DECREASE_STEP

    ELSE:
        difficulty = current_difficulty

2. // Adjust based on response speed
    IF response_time < FAST_THRESHOLD:
        difficulty = difficulty + INCREASE_STEP

    ELSE IF response_time > SLOW_THRESHOLD:
        difficulty = difficulty - DECREASE_STEP

3. // Clamp difficulty within bounds
    IF difficulty > MAX_DIFFICULTY:
        difficulty = MAX_DIFFICULTY

    IF difficulty < MIN_DIFFICULTY:
        difficulty = MIN_DIFFICULTY

4. RETURN difficulty
```

Architecture Diagram



Implementation steps and Execution

- **Step 1 — System Initialization & Environment Setup:** The Streamlit UI launches and the backend environment is configured. The Ollama server initializes to serve local LLMs including LLaMA, Phi, and Qwen. The FAISS and ChromaDB vector databases are connected and made ready for storage. The system automatically detects available GPU hardware via CUDA. Model selection and resource allocation are finalized before any user interaction begins.
- **Step 2 — Document Upload & Text Extraction:** The user uploads one or more academic PDF files through the interface. PyPDF2 processes each file and extracts text content page by page. Unnecessary formatting artifacts such as headers and footers are removed. The cleaned text is then converted into a structured format suitable for downstream processing.
- **Step 3 — Text Chunking & Preprocessing:** The extracted text is split into smaller segments using LangChain's RecursiveTextSplitter. Each chunk is approximately 1,000 tokens in size with a 200-token overlap between consecutive chunks. This overlap ensures that concepts spanning paragraph boundaries are not lost during retrieval. Every chunk is assigned a unique identifier and linked to its source document metadata.
- **Step 4 — Embedding Generation & Vector Storage:** Each text chunk is converted into a dense vector representation using Sentence-Transformer models. The all-mpnet-base-v2 model produces 768-dimensional vectors while all-MiniLM-L6-v2 produces 384-dimensional vectors for faster deployments. These embeddings are stored in a FAISS Inverted File Index structure. GPU acceleration is used to enable fast approximate nearest-neighbour search across the indexed chunks.
- **Step 5 — Knowledge Graph Construction:** The LLM analyses the document content and identifies key academic topics. Each topic is classified as a Fundamental Concept, Derived Concept, or Technique and Method. Dependency relationships between topics are mapped as directed edges using NetworkX. The Betweenness Centrality algorithm is then applied to identify the most structurally important topics in the graph.

Implementation steps and Execution

- **Step 6 — Learning Path Generation:** Kahn's Topological Sort algorithm is applied on the Knowledge Graph to produce a dependency-aware study sequence. Topics are ordered so that no concept appears before its prerequisites have been covered. Where multiple valid orderings exist, Betweenness Centrality scores are used as a tiebreaker. The resulting path is divided into sections grouping topics at the same conceptual depth.
- **Step 7 — Query Processing & Content Retrieval:** When a student selects a topic, it is converted into a vector embedding using the same sentence-transformer model. FAISS performs a semantic similarity search across the indexed chunks to find the most relevant content. The top-k matching chunks are retrieved and passed to LangChain, which constructs a structured prompt. The locally hosted Ollama LLM then generates a grounded explanation based entirely on the retrieved content.
- **Step 8 — Quiz Generation & Evaluation:** The Quiz Generator module receives the retrieved content chunks along with the current difficulty level. The LLM generates multiple-choice questions, each with one correct answer and three plausible distractors, formatted as structured JSON. Questions are presented to the student one at a time through the interface. Upon completion, the student's accuracy score and average response time are recorded for evaluation.
- **Step 9 — Mastery Tracking & Adaptive Learning:** The Mastery-Based Progression Algorithm updates each topic's mastery score using a weighted average of all quiz attempts. Topics are categorised as Mastered when the score reaches 80%, otherwise marked as In-Progress or To-Learn. The Adaptive Difficulty Algorithm then adjusts the complexity of the next quiz question based on the student's performance. A correct and fast answer increases difficulty, an incorrect answer decreases it, and re-engagement with study material is triggered if mastery is not yet achieved.
- **Step 10 — Visualization & Progress Monitoring:** Plotly renders the Knowledge Graph interactively using the Spring Layout algorithm, allowing students to explore topic relationships visually. The Progress Dashboard displays the student's overall session completion percentage, per-topic mastery scores, and topic type distribution. The most important topics are ranked and displayed according to their betweenness centrality scores. All visualizations and progress metrics update automatically after each completed study and quiz cycle.

Results obtained

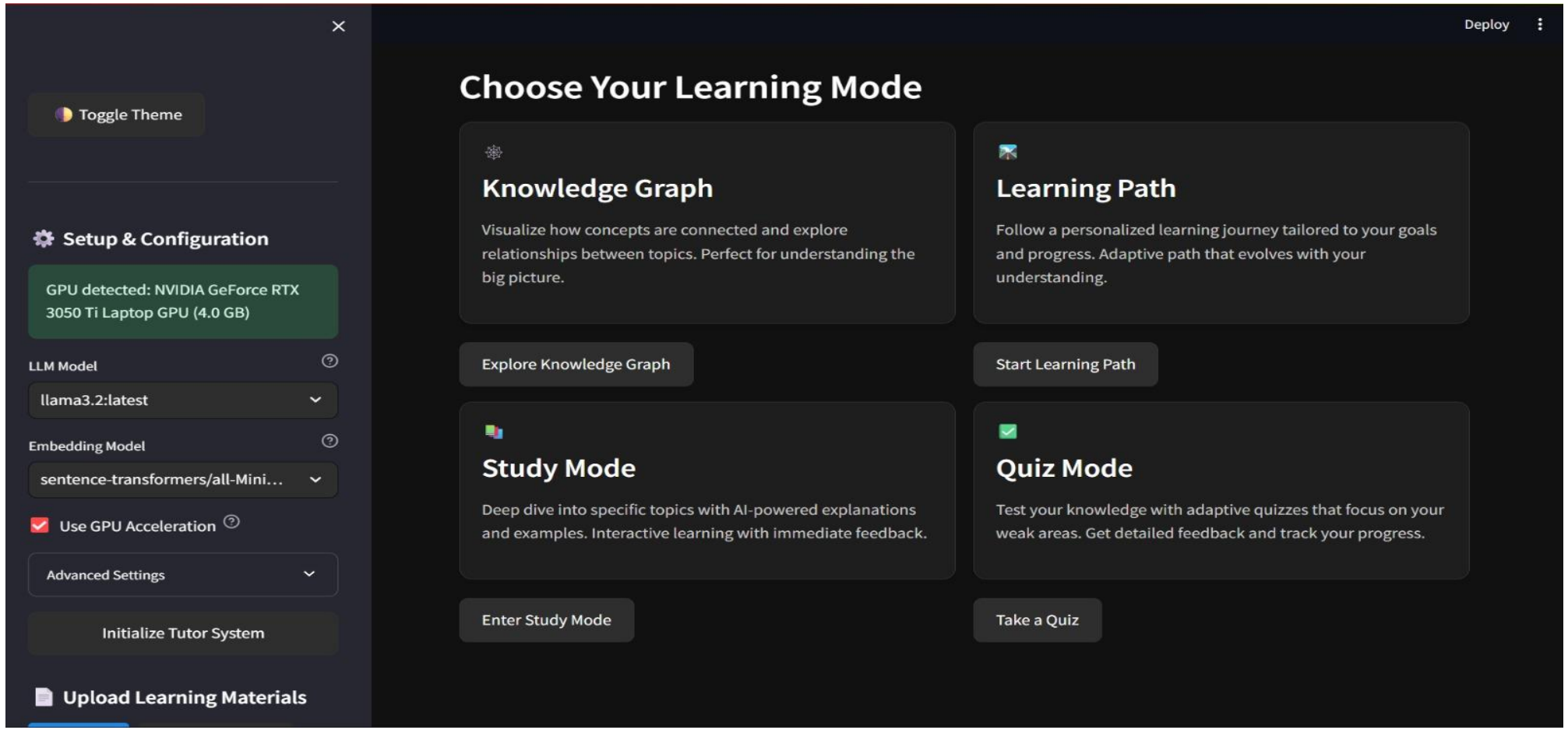


Figure 1: Main Dashboard – Learning Mode Selection Interface
The main dashboard presents four learning modes — Knowledge Graph, Learning Path, Study Mode, and Quiz Mode — as interactive cards. The sidebar confirms successful GPU detection and model initialization, indicating the system is ready for a student session

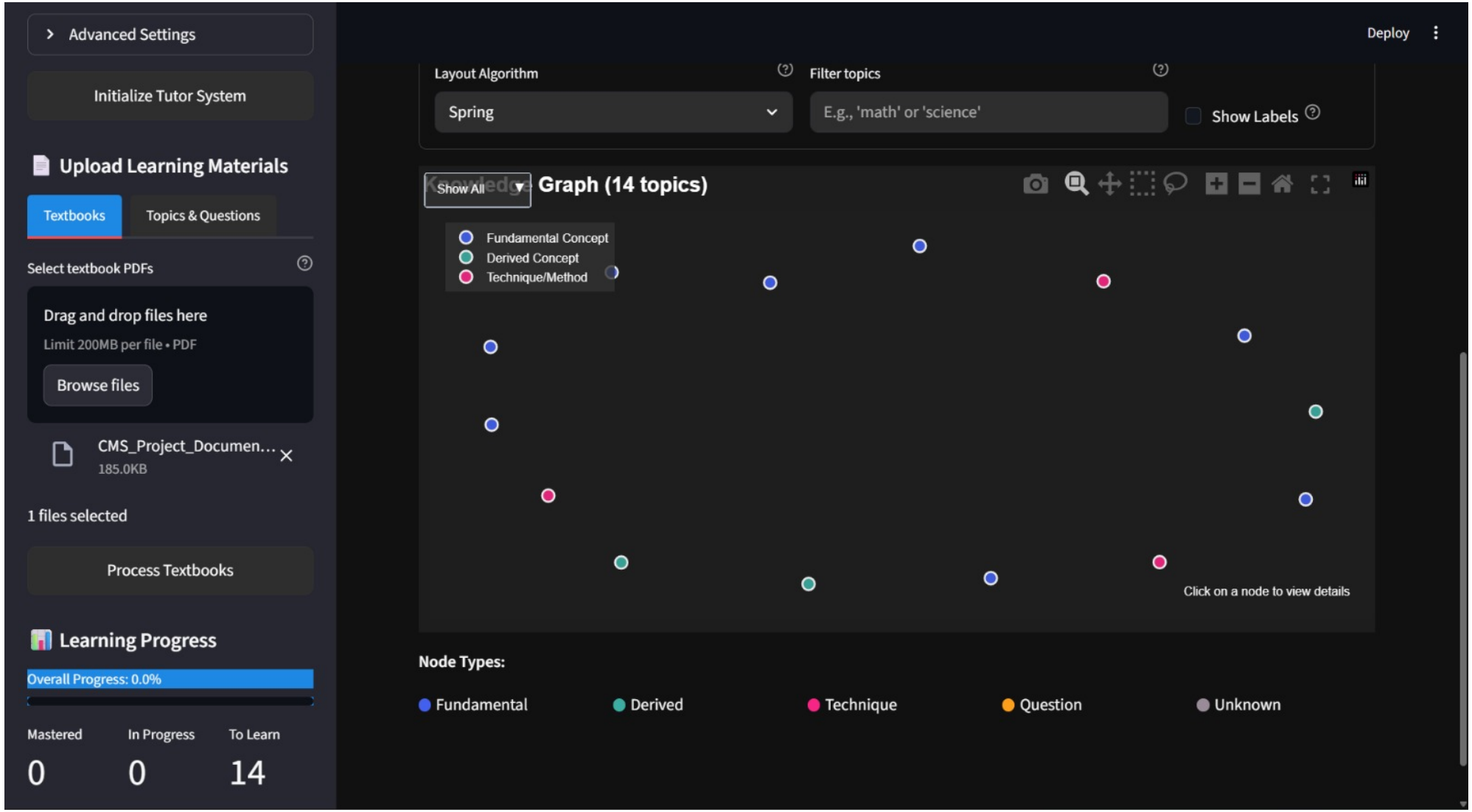


Figure 2: Knowledge Graph – Topic Visualization After Document Processing
After processing the uploaded PDF, the system constructed a Knowledge Graph of 14 topics, categorized into Fundamental Concepts (blue), Derived Concepts (green), and Techniques/Methods (red). The Spring Layout Algorithm distributes nodes spatially, with the sidebar tracking all 14 topics as "To Learn" at session start

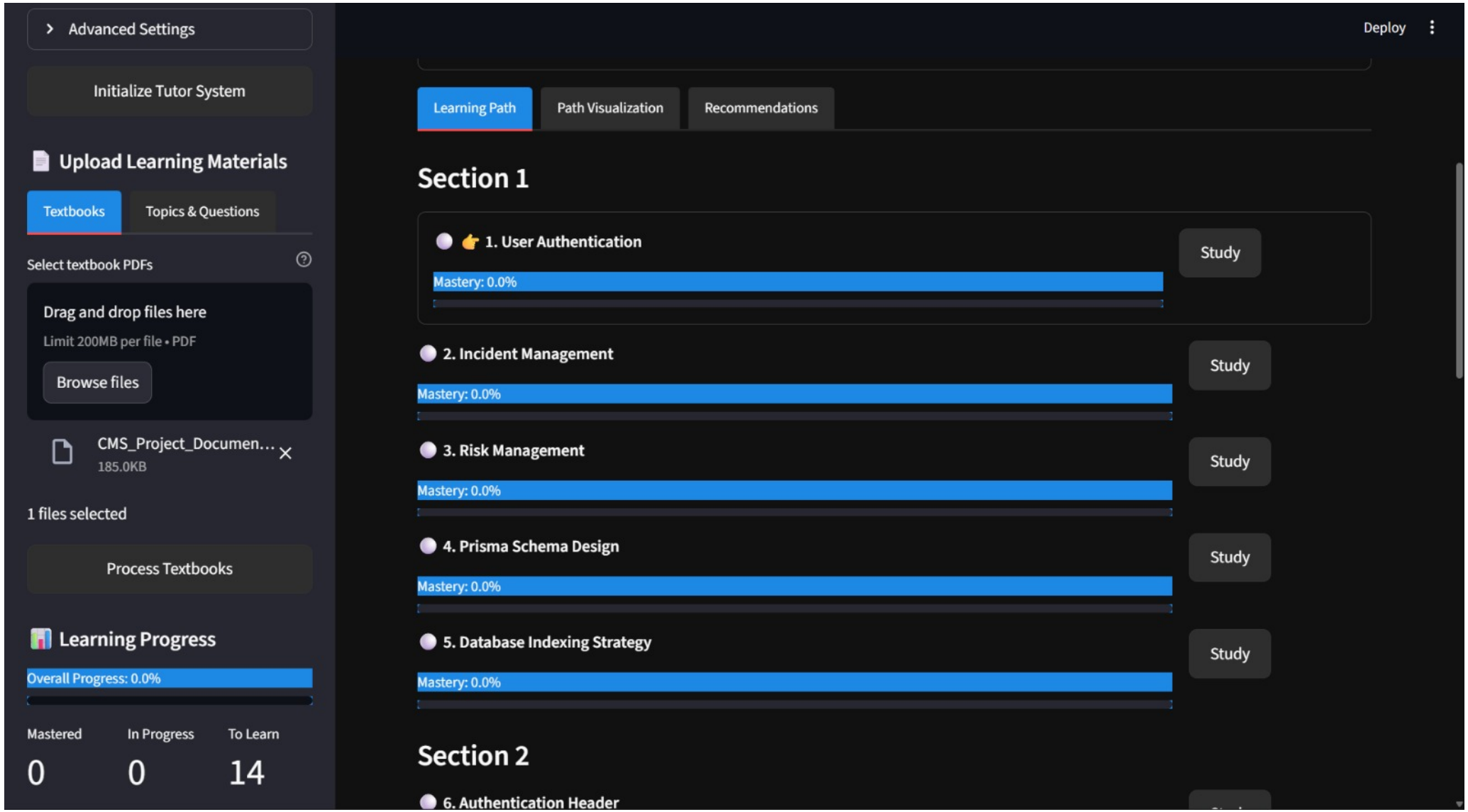


Figure 3: Learning Path – Topologically Sorted Study Sequence
The Learning Path displays topics in dependency order across two sections, with each topic showing a mastery progress bar at 0.0% and a dedicated "Study" button. This sequenced view ensures students encounter prerequisite concepts before advancing to more complex material

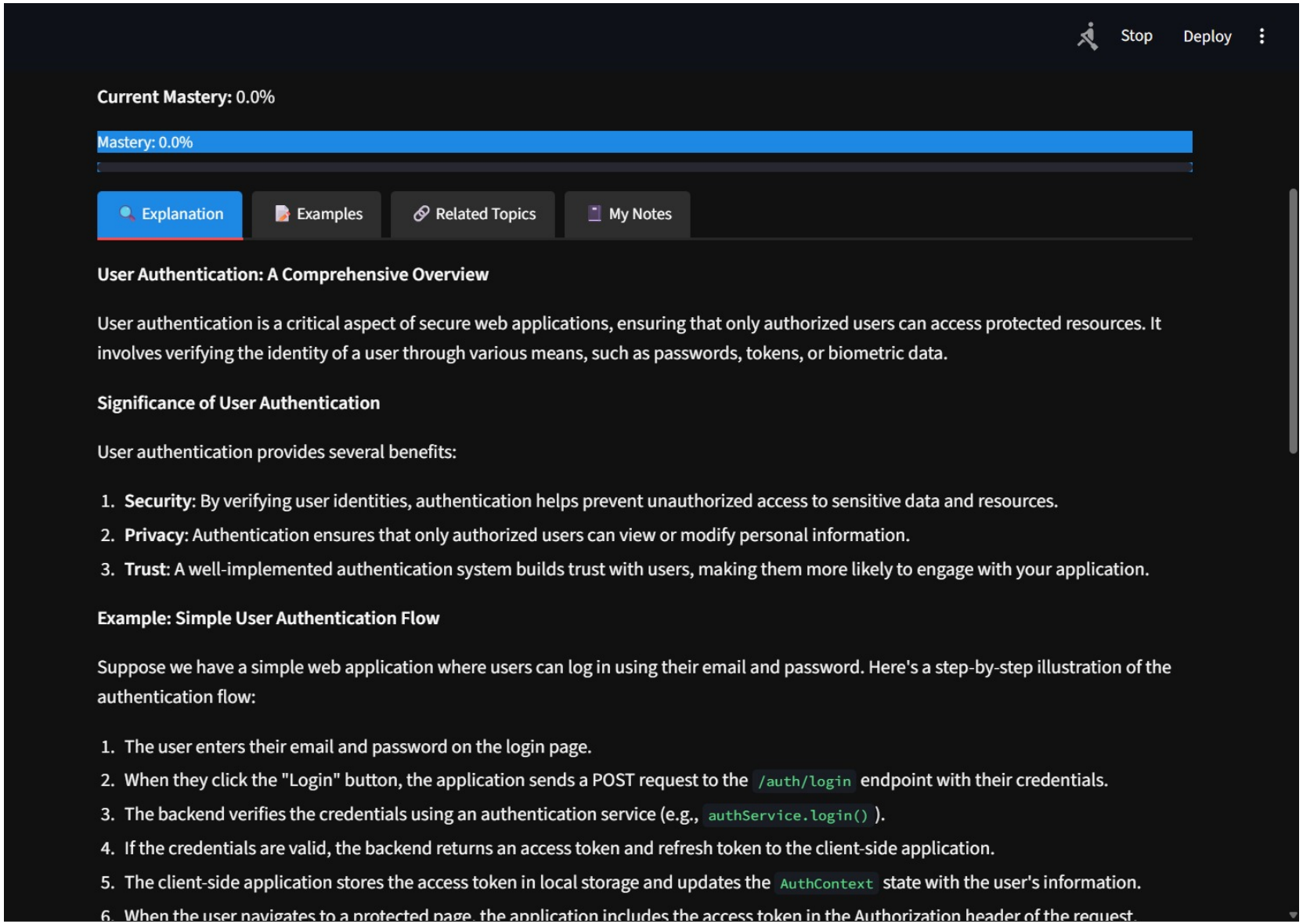


Figure 4: Study Mode – AI-Generated Explanation for "User Authentication"
The Study Mode interface presents an LLM-generated explanation for the selected topic, drawn entirely from the uploaded document via RAG retrieval. The content is organized under structured headings and delivered through four tabs: Explanation, Examples, Related Topics, and My Notes.

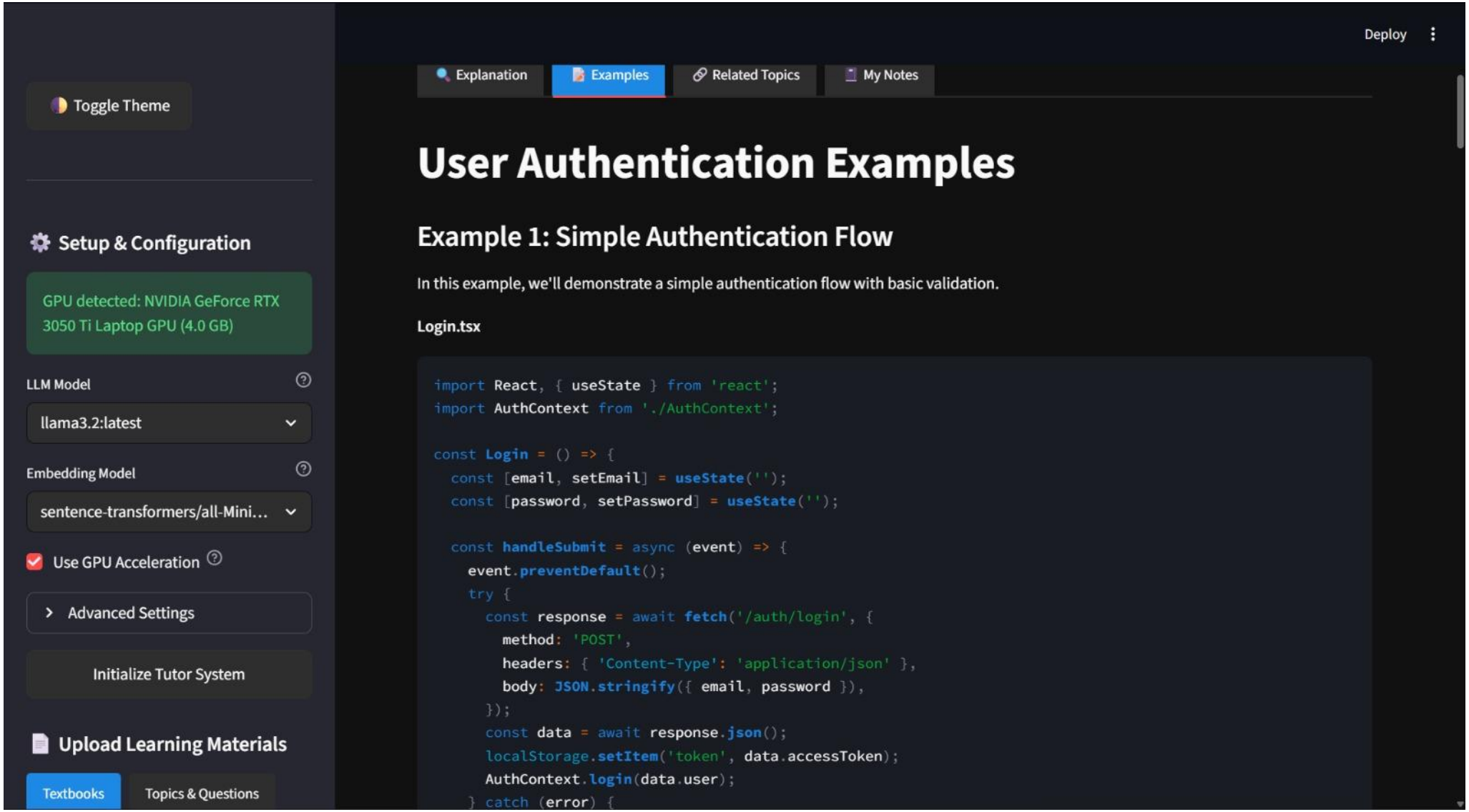


Figure 5: Study Mode – Code Example Generated for "User Authentication"
The Examples tab within Study Mode generates a practical, topic-relevant code demonstration — in this case, a Login.tsx React component illustrating a complete authentication flow. This confirms the system's ability to produce technically accurate, document-grounded code examples alongside conceptual explanations

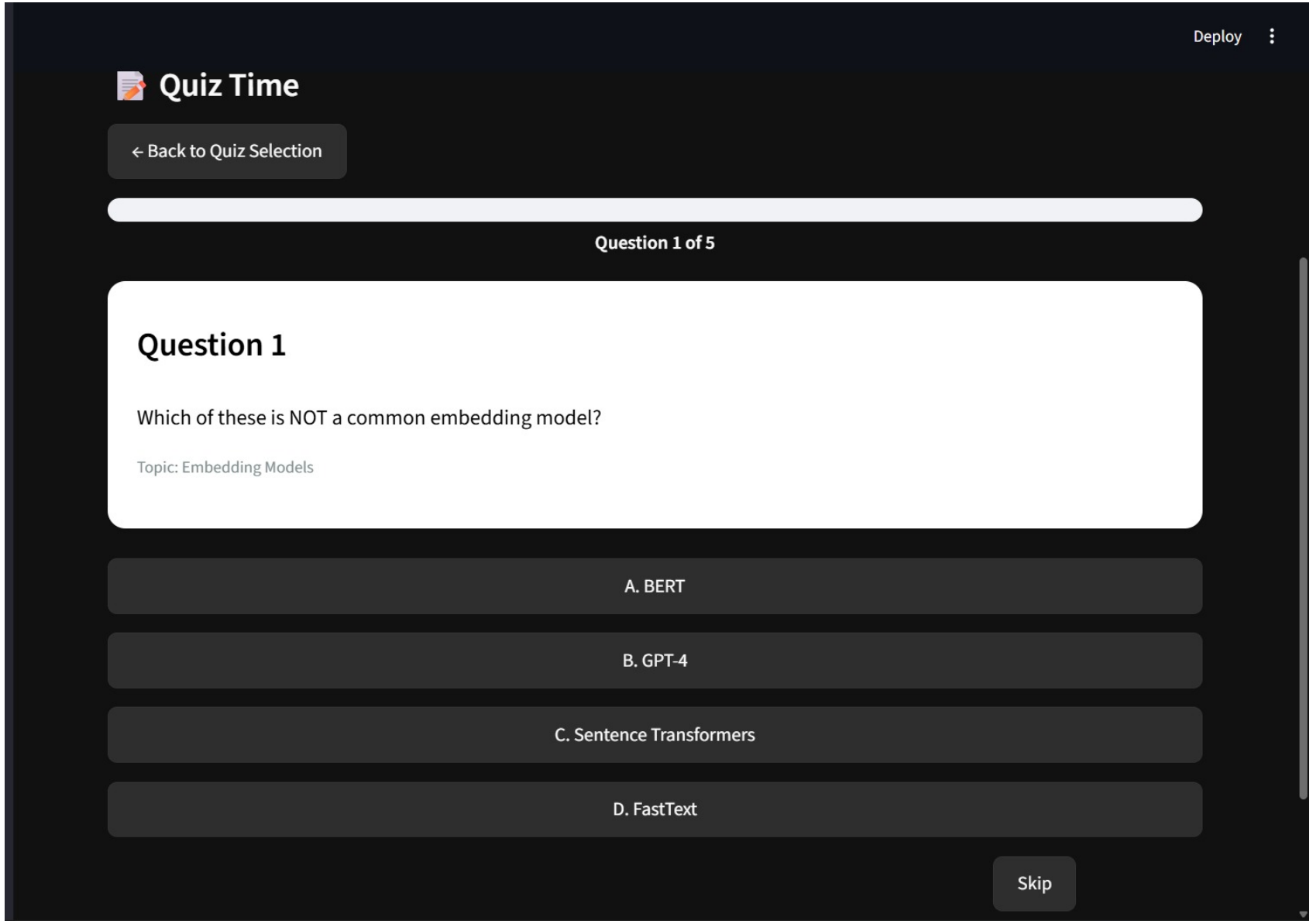


Figure 6: Quiz Mode – Adaptive Multiple-Choice Question Interface
The Quiz Mode presents one auto-generated multiple-choice question at a time, with a visible progress indicator (Question 1 of 5) and a Skip option. Questions are drawn from the same retrieved content used during study, ensuring direct alignment between what the student learned and what they are assessed on.

Results Analysis

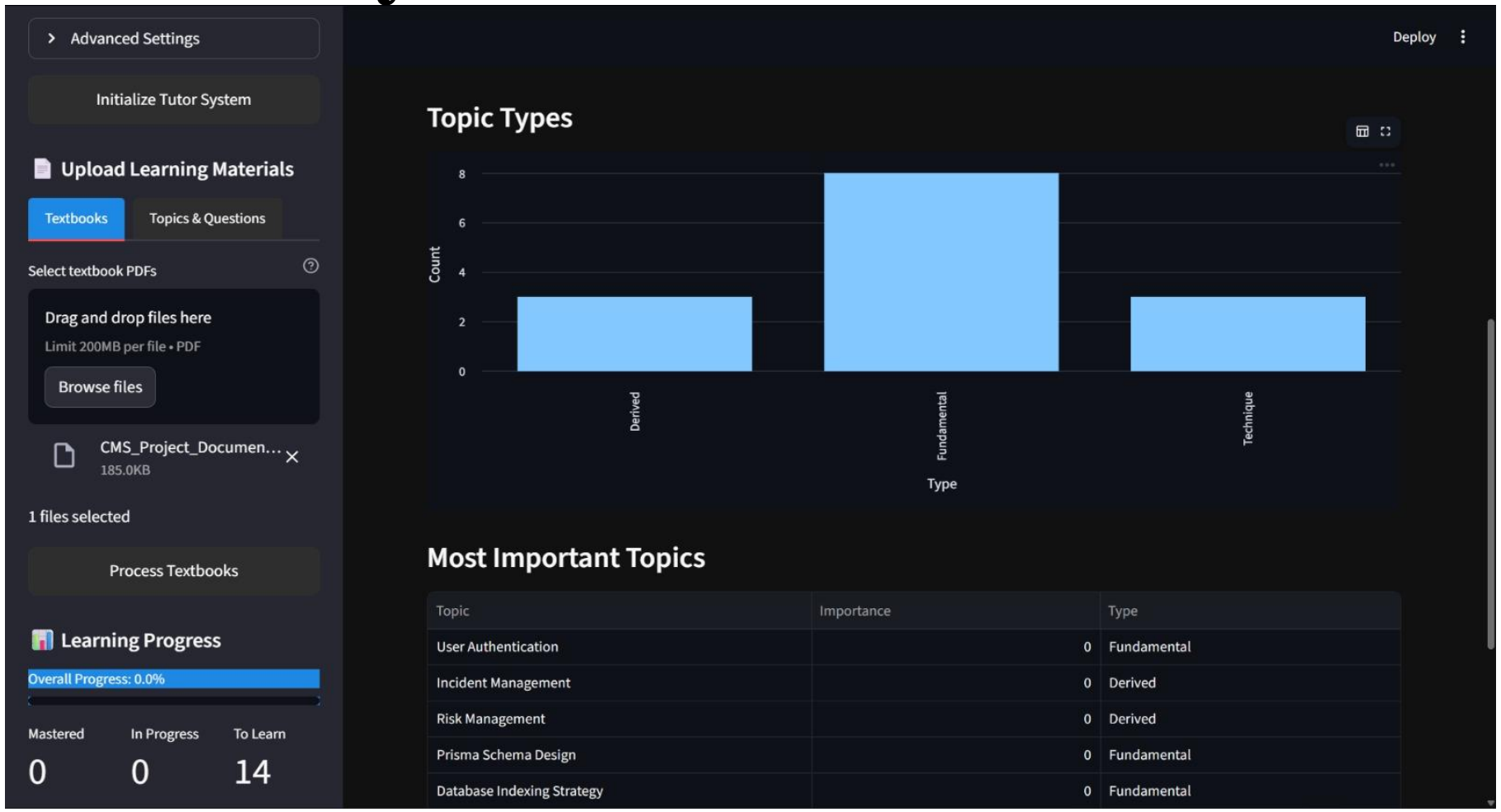


Figure 7: Topic Type Distribution Chart

The figure displays a bar chart breaking down the 14 extracted topics by type — Derived, Fundamental, and Technique — giving a quick visual summary of the document's conceptual composition. Below the chart, the Most Important Topics table ranks concepts by their betweenness centrality score, highlighting which topics serve as the most critical connectors in the knowledge structure.

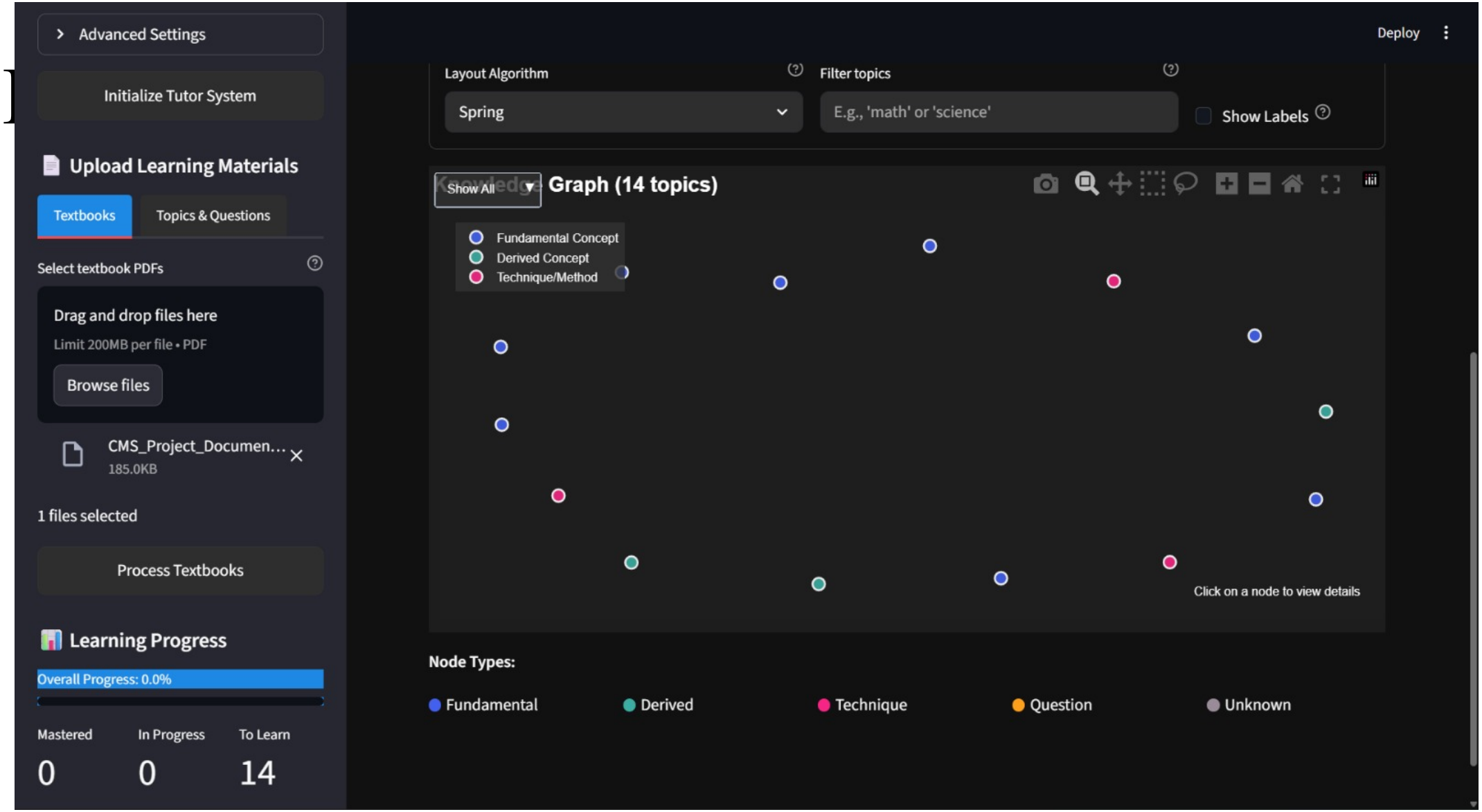


Figure 8: Knowledge Graph – Topic Visualization After Document Processing
After processing the uploaded PDF, the system constructed a Knowledge Graph of 14 topics, categorized into Fundamental Concepts (blue), Derived Concepts (green), and Techniques/Methods (red). The Spring Layout Algorithm distributes nodes spatially, with the sidebar tracking all 14 topics as "To Learn" at session start

Conclusion

SaraswathAI was developed to address a fundamental gap in digital education — the disconnect between content availability and actual knowledge acquisition. The platform evolved from a GPU-accelerated document retrieval system with blockchain-backed content verification into a fully agentic learning environment that autonomously manages the entire student learning lifecycle. In its final form, the system constructs a Knowledge Graph from uploaded academic documents, generates a dependency-aware learning path, delivers RAG-grounded topic explanations through locally hosted LLMs, and continuously assesses student understanding through adaptive quizzes — all without requiring external APIs or cloud dependencies. The result is a platform that does not merely answer questions but actively builds student understanding through a structured, self-correcting study-assess-progress cycle.

Future Work

- **Blockchain Re-integration:** The blockchain verification layer remains architecturally compatible with the current platform. Future work will link every AI-generated explanation and quiz question back to a cryptographically verified source, creating a tamper-evident learning record suitable for institutions where academic integrity and auditability are formal requirements.
- **Multi-Document Support and Longitudinal Tracking:** The system currently builds a Knowledge Graph from a single uploaded document. Extending this to support multiple documents simultaneously would create a unified curriculum-wide knowledge structure. Persisting learning state across sessions would further enable spaced repetition and long-term mastery tracking.
- **Richer Assessment and LMS Integration:** Future versions will introduce short-answer and code-based assessments to test deeper understanding beyond multiple-choice recall. Integrating with Learning Management Systems such as Moodle and Canvas through standard LTI protocols would allow SaraswathAI to operate as a seamless part of existing institutional academic workflows.

References

[1] Fang, Jinyuan, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. "A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems." arXiv:2508.07407 (2025).

[2] Author(s) Unknown. "LLM-Based Educational Content Retrieval and Intelligent Query Processing." *Information Processing & Management* (2026). Elsevier.

[3] Author(s) Unknown. "AI-Driven Learning Interaction and Engagement Framework for Digital Education." *Open Access Ed-Tech Journal* (2026). Elsevier.

[4] Author(s) Unknown. "Personalized Learning Pathways and AI-Guided Academic Improvement in Higher Education." *Nurse Education in Practice* (2026). Elsevier.

[5] Chen, Junyi, Zheyong Xie, Tong Xu, Sheng Zhou, Hongzhi Yin, and Enhong Chen. "Agent4EDU: Advancing AI for Education with Agentic Workflows." ResearchGate (2025).

[6] Veschetti, Adele, Richard Bubel, and Reiner Hähnle. "SmartML: Enhancing Security and Reliability in Smart Contract Development." *Blockchain: Research and Applications* (2025): 100362. Elsevier.

[7] Kulkarni, S., et al. (2024). "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models." ACL 2024.f

[8] Zhang, Y., et al. (2024). "A Comprehensive Survey of Retrieval-Augmented Generation: Evolution, Current Landscape and Future Directions." arXiv:2410.12837.

[9] Kulothungan, K., et al. (2025). "Blockchain-based AI Decision Audit Trail Framework for IoT Environments." IEEE Transactions on Industrial Informatics.

[10] European Union. (2024). "Regulation on Artificial Intelligence (AI Act)." Official Journal of the European Union.

Thank you…